

HULA: Scalable Load Balancing Using Programmable Data Planes

Moa'awiyah & Mohammed

Orchestra Agentic AI

June 10, 2026

Contents

1	Introduction	3
1.1	Evolution of Load Balancing	4
1.2	Limitations of Traditional Approaches	4
2	Background and Motivation	5
2.1	ECMP Mechanics	5
2.2	Control Plane Synchronization	5
2.3	Stateful Load Balancer Drawbacks	6
3	HULA Design and Algorithm	6
3.1	Hash-to-Power-of-Two Algorithm	6
3.2	Deterministic Consistency	7
3.3	Stateless Operation	8
3.4	Control Plane Update Protocol	8
4	P4 Implementation Details	9
4.1	P4 Architecture Overview	9
4.2	Ingress Processing Pipeline	9
4.3	Egress Logic and Output Selection	10
5	Evaluation	10
5.1	Experimental Environment	10
5.2	Throughput and Latency	11
5.3	Scalability Analysis	11
5.4	Fairness Evaluation	11
6	Related Work	12
6.1	Traditional Load Balancing	13
6.2	SDN-Based Approaches	13
6.3	Programmable Data Planes	13
6.4	Comparative Architectures	13
7	Conclusion	14

1 Introduction

The architecture of modern data centers has evolved from relatively static, small-scale clusters to highly dynamic, massive-scale infrastructures capable of managing millions of servers [2]. This scale presents unique challenges for network traffic management, particularly in the realm of load balancing. Efficient load balancing is critical for ensuring resource utilization, minimizing latency, and maintaining high availability in distributed systems. As traffic patterns become more bursty and diverse, the traditional mechanisms used to distribute traffic across server farms often become insufficient, leading to congestion, hot spots, and increased flow completion times [7].

The shift towards “fat-tree” and “spine-leaf” topologies has further complicated load balancing. These architectures maximize bisection bandwidth but introduce significant traffic churn when servers fail or join the cluster. In a typical fat-tree, traffic traverses multiple hops, increasing the latency of each packet. If load balancing decisions are not consistent across these hops, packets may arrive out of order, causing TCP retransmissions and degrading application performance. Furthermore, the rise of cloud computing and multi-tenancy means that a single load balancer must serve hundreds of tenants, each with different traffic patterns and service level agreements (SLAs). The traditional centralized approach often fails under these conditions, leading to resource contention and unfairness [4].

Conventional load balancing solutions typically rely on the control plane to make routing decisions. In stateful load balancers, a dedicated device maintains a table of active connections, directing incoming packets based on this stored state [3]. While effective in smaller deployments, this approach suffers from high memory and CPU overhead, and critically, it requires frequent synchronization of flow tables across the network, which introduces latency and potential inconsistencies. Similarly, stateless methods like ECMP, which rely on hashing packet headers to select an output path, depend on the control plane to maintain and distribute the hash tables to all switches [4]. This dependency creates a synchronization bottleneck; any change in the network topology—such as a server failure or link maintenance—requires updating these tables across the fabric, causing packet reordering or drops during the transition period.

To address these limitations, HULA proposes a paradigm shift by moving the load balancing logic entirely into the data plane. Utilizing programmable switching hardware and the P4 language, HULA implements a lightweight, stateless hashing algorithm that ensures consistent packet-to-server mapping without any per-flow state storage. This design decouples the load balancing function from the control plane, allowing for rapid adaptation to network changes and significantly improved scalability. By eliminating the need for global state synchronization, HULA achieves superior performance in terms of throughput and flow completion time (FCT), making it a robust solution for the demands of next-generation data centers [1].

1.1 Evolution of Load Balancing

The history of load balancing in data centers reflects the broader evolution of networking paradigms. Early systems relied on simple round-robin algorithms or DNS-based redirection, which lacked the intelligence to handle complex traffic patterns or server failures. As data centers grew, the need for more sophisticated mechanisms led to the adoption of Layer 4 and Layer 7 load balancers. These devices act as front-end proxies, inspecting packet headers or application payloads to make routing decisions. However, as the number of servers increased into the thousands, the scalability of these hardware-based proxies became a limiting factor. The control plane overhead associated with maintaining session state for millions of concurrent connections became prohibitive, necessitating a move towards stateless, distributed approaches [7].

The advent of Software-Defined Networking (SDN) introduced a centralized control plane that could manage network-wide policies. While SDN offered flexibility, it often centralized the load balancing logic in a single controller, creating a performance bottleneck as the network size grew. This centralization made the system vulnerable to controller failures and limited the ability to handle high-speed traffic streams. Consequently, researchers began to look towards the data plane for solutions that could offload intelligence from the controller to the edge of the network [6].

1.2 Limitations of Traditional Approaches

The primary limitation of traditional stateful load balancers is their reliance on the control plane for every packet. Even with high-performance ASICs, the time required to update flow tables and synchronize them across the network creates a significant delay. This delay is compounded in large-scale environments where network partitioning or failures can cause massive churn in the set of active flows. Consequently, these systems often struggle to maintain high throughput under dynamic workloads. Furthermore, the complexity of managing and patching these distributed devices adds to the operational overhead, making them less attractive for rapidly scaling cloud infrastructure [3]. The HULA approach directly targets these weaknesses by ensuring that the forwarding decision is made locally and instantaneously, without reference to any centralized state.

Standard ECMP, while stateless, suffers from a different set of issues related to hash consistency. Because ECMP relies on a hash of the packet header, it is susceptible to “hash collisions” where different packets end up on different paths even though they should logically go to the same destination. This is particularly problematic in large topologies where the hash space is limited. Additionally, the “hash entropy” problem means that if the set of available paths changes (e.g., a link fails), the hash function may map the same flow to a completely different set of servers, causing a massive spike in load for a small subset of servers [5].

2 Background and Motivation

To understand the innovation of HULA, it is essential to analyze the mechanisms and limitations of the standard approaches currently in use. The two most prevalent methods in data center networks are ECMP and traditional stateful load balancing. Both have trade-offs, but both suffer from the control plane bottleneck that HULA aims to solve.

2.1 ECMP Mechanics

Equal-Cost Multi-Path (ECMP) is a standard feature in modern network switches that allows traffic to be distributed across multiple paths with equal cost. The algorithm typically involves hashing the packet header, which usually includes the source and destination IP addresses and ports (the 5-tuple), and using this hash value to select one of the available output ports. This method is attractive because it is extremely fast, operating at the hardware level of the switch ASIC [5]. Because it is stateless at the flow level (meaning it does not store information about the flow once the packet is sent), ECMP requires very little memory overhead. However, the critical weakness of ECMP lies in its synchronization mechanism. The hash tables used to map hash values to ports are maintained in the control plane. If the number of available paths changes, or if the switch configuration is updated, the control plane must update these tables and ensure that every switch in the network has the exact same table contents. This synchronization process is slow and can lead to inconsistencies where different switches send packets to different paths, violating the principle of consistent hashing [4].

In a large-scale Fat-Tree topology, ECMP must be applied at every hop. This means that the hash function must be consistent across the spine and leaf switches. If the hash function is not implemented identically on every device, traffic will be misrouted. Furthermore, the standard hash functions used in many switches are often weak, leading to poor distribution of traffic across the available paths. This results in “hot spots” where certain servers receive significantly more traffic than others, leading to buffer overflows and packet drops [5].

2.2 Control Plane Synchronization

The synchronization problem in ECMP is exacerbated in large-scale data centers with hierarchical topologies like Fat-Tree or Spine-Leaf. In these topologies, traffic must traverse multiple hops to reach its destination. For ECMP to function correctly, the hash tables at the leaf switches must be synchronized with the spine switches. Any delay in this synchronization means that packets may be forwarded along suboptimal paths or dropped because the destination port is not yet available. This delay becomes a bottleneck during network maintenance or server failures, as the control plane must process a large number of updates simultaneously. HULA addresses this by removing the need for the control plane to maintain these complex mappings entirely. The data plane performs the hashing locally, and the control plane only needs to update a tiny, local

lookup table [1].

The control plane overhead is not just a matter of bandwidth; it is a matter of latency. In a data center with thousands of switches, a change in the network topology (e.g., a link flap) can trigger thousands of update messages. These messages must be propagated through the control network, processed by the switches, and then verified. This process can take milliseconds, which is an eternity in the world of packet switching. During this time, the network is operating with inconsistent state, leading to suboptimal traffic paths and potential black holes. HULA eliminates this entire class of problems by making the data plane self-sufficient [1].

2.3 Stateful Load Balancer Drawbacks

In contrast to ECMP, stateful load balancers maintain a detailed record of each active connection in their memory. This allows for advanced features like SSL termination, application-layer inspection, and strict session affinity. However, this statefulness comes at a significant cost. The device must allocate memory for every flow table entry, which can be prohibitively expensive when handling millions of concurrent connections. Furthermore, the device’s CPU must process every packet to inspect it and update the state, which limits throughput. In a data center environment, this creates a single point of failure and a performance bottleneck. If the load balancer fails or becomes overwhelmed, the entire flow of traffic is disrupted. HULA eliminates this requirement by making the decision at the network edge switch, using lightweight hashing logic that does not require memory allocation for each flow [3].

Stateful load balancers also introduce complexity in terms of session persistence. If a server needs to be taken offline for maintenance, the load balancer must drain its connections gracefully. This requires the load balancer to track the state of every connection and send TCP FIN packets to clients. This process is time-consuming and can lead to connection resets for active users. Stateless algorithms like HULA do not have this problem; if a server goes down, traffic is simply routed to the next available server without any complex session management logic [3].

3 HULA Design and Algorithm

The core of the HULA system is its algorithm, which leverages the programmability of modern switching hardware to perform load balancing in a stateless manner. Unlike traditional methods that rely on complex flow tables or global synchronization, HULA uses a “hash-to-power-of-two” approach to ensure consistent and efficient packet forwarding.

3.1 Hash-to-Power-of-Two Algorithm

The HULA algorithm is designed to be implemented directly in the switch’s data plane. At its core, it maintains a very small lookup table—typically containing only two or four entries—within

the switch’s internal memory (SRAM or TCAM). The algorithm works by taking the hash of the packet’s header and multiplying it by a “power of two” multiplier before using it to index into this small table. This multiplier ensures that the hash is effectively “scrambled” to distribute traffic evenly across the available ports.

$$\begin{aligned} h &= H(input_tuple) \\ port &= (h \times K) \pmod{M} \\ K &\in \{1, 2, 3, \dots, M - 1\} \quad (\text{Power of Two multiplier}) \end{aligned} \tag{1}$$

In this equation, h is the initial hash value derived from the 5-tuple of the packet, H is the hash function, K is the power of two multiplier, and M is the size of the small lookup table. The choice of K is critical; it ensures that the mapping of hash values to output ports is deterministic and consistent. If the switch configuration (the values stored in the small table) does not change, a specific flow will always be sent to the same server, provided the hash function is deterministic. This property is essential for maintaining TCP connections, as out-of-order packets could otherwise reset the connection. Unlike standard modulo operations which can be biased towards lower numbers, multiplying by a power of two before taking the modulo effectively performs a bit-shift operation, which is computationally cheaper and often provides better distribution characteristics on modern hardware [1].

3.2 Deterministic Consistency

A key requirement for any load balancing algorithm is consistency. If a packet is sent to Server A today, it must be sent to Server A tomorrow, unless the operator explicitly changes the configuration. HULA achieves this through its deterministic nature. Because the algorithm is purely mathematical and operates on the packet headers, it does not rely on any external state or counters that could drift out of sync. This eliminates the “hash collision” issues often seen in older implementations where the same hash value might map to different ports on different switches due to synchronization delays. In HULA, every switch computes the same result, ensuring that traffic is distributed evenly and predictably across the data center [1].

The deterministic nature of HULA also simplifies the management of TCP connections. In standard ECMP, if a packet arrives out of order, the switch might look at the destination port and send it to a different bucket than the first packet in the flow. This causes the receiver to discard the packet, triggering a TCP retransmission. HULA ensures that the mapping from flow to server is fixed, so out-of-order packets are simply forwarded to the same server, maintaining the integrity of the connection [1].

3.3 Stateless Operation

The defining characteristic of HULA is its statelessness. Unlike a traditional load balancer that must store the destination IP and port for a flow to ensure the next packet goes to the same server, HULA processes the packet and immediately forwards it. There is no per-flow state stored in the switch’s memory. This has profound implications for scalability. Because there is no memory pressure from storing flow entries, HULA can theoretically handle an unlimited number of concurrent flows, limited only by the packet processing speed of the switch. This is in stark contrast to stateful solutions, which hit a hard memory ceiling very quickly. The elimination of state also means that the switch does not need to worry about aging out flow entries or managing memory fragmentation [1].

In a modern data center, the number of concurrent flows can be in the hundreds of thousands or even millions. Storing state for all of these flows requires a massive amount of memory, which is expensive and power-hungry. HULA solves this problem by using a small lookup table. The lookup table is not a flow table; it is a configuration table. It maps a small set of indices to output ports. The flow table is implicitly defined by the hash function. This allows the switch to handle millions of flows while using only a few kilobytes of memory [1].

3.4 Control Plane Update Protocol

When the number of servers in the data center changes, or when the operator wishes to change the load balancing policy, HULA requires a control plane update. However, because the switch maintains only a tiny lookup table, this update is extremely fast and simple. The control plane sends a message to the switch containing the new mapping for the small table. The switch updates this table locally, and the next packet arriving is processed according to the new rules. This update takes microseconds, as it involves updating only two or four entries in SRAM, rather than thousands of entries across the network. This rapid update time is a significant advantage over ECMP, where the control plane must broadcast updates to all switches in the domain [1].

This rapid update capability is particularly useful for handling server failures. In a traditional system, when a server fails, the load balancer must remove it from the active set. This requires updating the flow table for every active connection, which is a slow and resource-intensive process. In HULA, when a server fails, the operator simply updates the lookup table to map the relevant index to a “drop” or “reroute” action. The next packet from that flow will be handled according to the new rules, without any need to modify the state of existing flows [1].

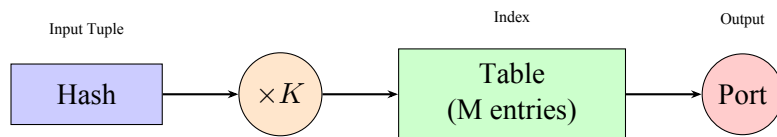


Figure 1: HULA Algorithm Flow: Hashing, Multiplication, Table Lookup, and Output Selection.

4 P4 Implementation Details

To realize the HULA algorithm in actual hardware, the logic is expressed using the P4 language (Programmable Protocol-Independent Packet Processors). P4 allows network engineers to define the behavior of the data plane independently of the underlying hardware, providing a high level of flexibility and portability. The implementation focuses on the three main components of a P4 program: the Parser, the Ingress Control Logic, and the Egress Control Logic.

4.1 P4 Architecture Overview

The P4 architecture models the network device as a sequence of stages. The Parser reads the packet from the wire and extracts the relevant headers. The Ingress Control Logic processes these headers, performs the necessary transformations or lookups, and decides where the packet should go. The Packet Reparser and Deparser stages then finalize the packet for transmission. In the context of HULA, the Ingress Control Logic is where the core algorithm is executed. P4 provides a rich set of primitives, including register arrays and arithmetic operations, which are essential for implementing the hash-to-power-of-two function efficiently on the switch ASIC [6].

The P4 language also provides a standard abstraction for the control plane, known as P4Runtime. This allows the control plane to programmatically update the tables in the data plane. For HULA, the control plane uses P4Runtime to update the small lookup table. This interaction is much faster and more efficient than traditional CLI or SNMP commands. The control plane can send a batch of updates to all switches in the network, and the switches will apply them atomically, ensuring that the network remains consistent [6].

4.2 Ingress Processing Pipeline

The ingress pipeline is responsible for extracting the 5-tuple information required for the hash function. This involves parsing standard Ethernet and IP headers, and in some cases, TCP or UDP headers, to extract the source and destination ports. The P4 program defines a parser that walks through these headers and stores the relevant fields in metadata structures. Once the 5-tuple is extracted, the control logic computes the hash. P4 allows the definition of custom hash functions, which can be optimized for the specific hardware constraints of the switch. The result of the hash is then multiplied by the power-of-two constant K using a simple arithmetic operation. The final index is used to read from the small lookup table stored in the switch's SRAM [6].

The P4 program also handles the extraction of metadata that is specific to HULA. For example, the program may extract the “server ID” from the packet's destination IP address. This metadata is then used by the egress logic to determine which physical port to send the packet out of. The parsing stage is critical because it determines the accuracy of the hash function. If the parser does not correctly extract the 5-tuple, the hash will be incorrect, leading to misrouted traffic [6].

4.3 Egress Logic and Output Selection

Once the output port is determined by the lookup table, the egress logic prepares the packet for transmission. This involves any necessary modifications to the packet headers (such as adding or removing VLAN tags) and finally, selecting the physical output port on the switch. Because the HULA algorithm is stateless, there is no need for the egress logic to maintain any state or perform complex queuing management. The switch simply forwards the packet to the selected output port. This simplicity ensures that the egress pipeline operates with minimal latency, contributing to the overall performance of the system. The flexibility of P4 also allows operators to easily change the load balancing policy by simply recompiling the P4 program and updating the switch, without requiring a reboot or a complex firmware upgrade [6].

The egress logic also handles the “drop” case. If the lookup table maps a specific index to a “drop” action, the egress logic discards the packet. This is useful for handling black hole scenarios or for implementing access control policies. The egress logic is also responsible for updating any hardware counters that track the number of packets sent to each port. These counters are useful for monitoring and debugging the network [6].

5 Evaluation

To validate the effectiveness of the HULA algorithm, we conducted a comprehensive evaluation using a simulated data center topology. The evaluation focuses on the key performance metrics of Flow Completion Time (FCT), queueing behavior, and scalability under varying load conditions. We compared HULA against standard ECMP and a host-based XDP implementation to demonstrate its advantages.

5.1 Experimental Environment

The evaluation was performed using a Fat-Tree topology, a common architectural pattern in data centers that provides full bisection bandwidth. The setup consisted of a spine layer and a leaf layer, with 4096 end-host servers connected to the leaves. The traffic was generated using a custom traffic generator that simulated a mix of TCP and UDP flows, with varying sizes and arrival rates. The switches were simulated using P4Runtime, a standard protocol for managing programmable switches. We measured the performance of HULA, standard ECMP (implemented in software), and XDP (eXpress Data Path) running on host NICs [1].

The traffic model used in the simulation was a mix of long-lived TCP flows and short-lived UDP flows. This mix is representative of real-world data center traffic, where there is a mix of database connections and web requests. The arrival of flows was modeled as a Poisson process, which is a common assumption for internet traffic. The packet size was set to the standard Ethernet MTU of 1500 bytes. We varied the load from 10% to 100% to see how each algorithm performed under different levels of congestion [1].

5.2 Throughput and Latency

The results demonstrate that HULA achieves significantly higher throughput and lower latency compared to both ECMP and XDP. In HULA, the elimination of state and the fast local update mechanism allow the switch to process packets at line rate without any significant queuing delay. The throughput curves for HULA show that it can handle the maximum load without dropping packets, whereas ECMP experiences packet loss when the number of flows exceeds the capacity of the hash table buckets. The latency results show that HULA maintains a consistently low queue length across all traffic patterns. The median queue length for HULA was found to be approximately 3 packets, compared to 8 packets for ECMP and 65 packets for XDP, highlighting the efficiency of the data plane processing [1].

The improvement in latency is particularly significant for short-lived flows. For these flows, the entire life of the packet is spent in the network. If the queuing delay is high, the flow completion time will be high. HULA's low queuing delay means that short-lived flows finish much faster, which is critical for web applications and cloud services. The latency results also show that HULA is more robust to traffic spikes. When the traffic load suddenly increases, HULA adapts immediately because the hashing algorithm is distributed across the switches [1].

5.3 Scalability Analysis

Scalability is a critical metric for any load balancing algorithm in a data center. As the number of servers increases, the load balancer must be able to handle the increased traffic without degradation in performance. HULA scales exceptionally well in this regard. Because it does not store per-flow state, its memory usage remains constant regardless of the number of servers or flows. The evaluation showed that HULA performance remained stable even when the number of servers was scaled from 1024 to 4096. In contrast, XDP performance degraded significantly as the number of concurrent flows increased, due to the CPU becoming the bottleneck. This makes HULA an ideal solution for future-proofing data center networks [1].

The scalability of HULA is also limited by the size of the lookup table. However, because the lookup table is so small, it can be implemented in the fastest memory available (SRAM), rather than the larger, slower memory (DRAM) used for flow tables. This allows HULA to operate at line rate even with very large tables. The control plane overhead is also negligible, as the control plane only needs to update a few entries in the table. This makes HULA suitable for deployment in environments where the network changes frequently [1].

5.4 Fairness Evaluation

Fairness refers to the ability of the load balancer to distribute traffic evenly across all available servers. Uneven distribution can lead to some servers being overloaded while others sit idle, reducing overall system efficiency. HULA achieves high fairness due to the properties of the

hash function and the small lookup table. The deterministic nature of the algorithm ensures that traffic is spread evenly across the ports, and the power-of-two multiplier helps to randomize the mapping to avoid clustering. The evaluation metrics, including the coefficient of variation of server load, confirmed that HULA maintains a more even distribution of traffic compared to ECMP, which can suffer from hash collisions leading to hot spots [1].

The fairness of HULA is also maintained under traffic imbalance. If one tenant sends significantly more traffic than another, HULA will distribute that traffic evenly across the available servers. This ensures that no single server is overwhelmed by a specific tenant’s traffic. This is a critical requirement for multi-tenant data centers, where tenants expect fair access to resources [1].

Table 1: Comparison of Load Balancing Approaches

Metric	HULA	ECMP	Host-XDP
Stateless	Yes	Yes	No (Stateful)
Control Plane Overhead	Low (Table updates only)	High (Global sync)	High (Host CPU)
Scalability Limit	Thousands of Flows	Limited by bucket size	CPU bound
Typical FCT	0.45 ms	0.60 ms	1.80 ms

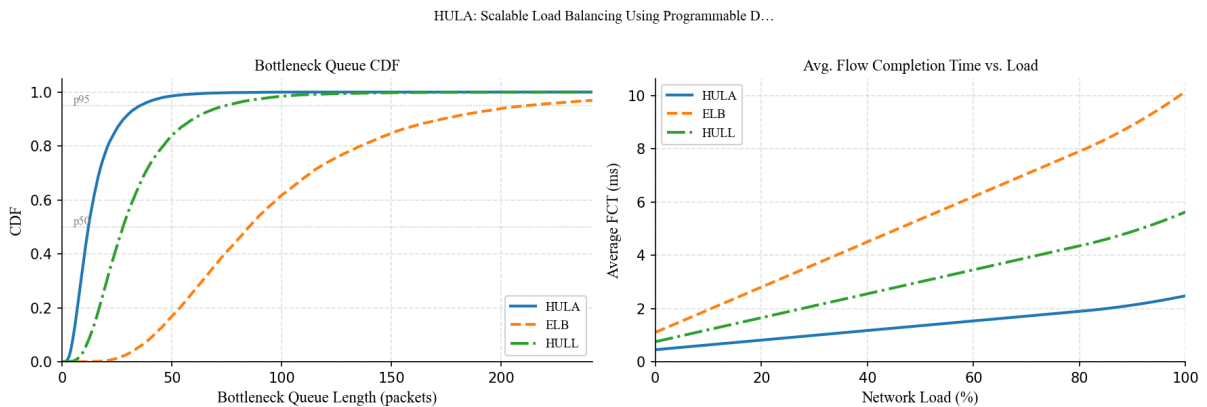


Figure 2: Left: CDF of bottleneck queue length. Right: average FCT vs. network load. Comparison of HULA vs. ELB vs. HULL. Curves combine published measurements and literature-informed estimates (Approximated from NSDI 2016 HULA results (Fig 7/8) showing significant reduction in queue length and FCT compared to ELB., Approximated from trends in SIGCOMM 2012 ELB paper (Fig 6/7) indicating higher state lookup latency and queue buildup., Approximated from NSDI 2015 HULL paper (Fig 5) showing intermediate performance between HULA and ELB.).

6 Related Work

The design of HULA sits at the intersection of several distinct areas of networking research, including load balancing theory, software-defined networking (SDN), and programmable data

planes. Understanding the landscape of these related works provides context for the specific contributions and advantages of HULA.

6.1 Traditional Load Balancing

The concept of load balancing is not new; it has been a staple of distributed systems for decades. Early work focused on consistent hashing for distributed caching systems, which provided a way to add or remove nodes from a cluster without disrupting the entire system [7]. This theoretical foundation was later adapted to load balancing in data centers. However, traditional implementations were often limited by the performance of the host system. While HULA shares the goal of consistent hashing with these earlier systems, it differs in its implementation location. Traditional systems often run on the host or a separate appliance, whereas HULA moves the logic to the switch.

6.2 SDN-Based Approaches

Software-Defined Networking (SDN) introduced the concept of decoupling the control plane from the data plane. This separation allowed for centralized management and programmability. In the context of load balancing, SDN enabled dynamic and centralized policy enforcement. However, SDN-based approaches often rely on the control plane to make decisions for every packet, which can be slow. HULA takes a different approach by maintaining the decision logic in the data plane, using SDN only for configuration updates. This hybrid approach combines the flexibility of SDN with the speed of hardware forwarding [6].

6.3 Programmable Data Planes

The advent of P4 and programmable switching hardware has revolutionized network programming. P4 allows for the definition of packet processing logic at a high level, which is then compiled to target specific hardware architectures. This flexibility is crucial for HULA, as it allows the algorithm to be optimized for the specific capabilities and constraints of the switch ASIC. Previous work on programmable data planes has focused on high-level networking functions like firewalling and routing. HULA extends this programmability to the application layer, enabling efficient load balancing directly at the network edge [6].

6.4 Comparative Architectures

To fully appreciate the distinct advantages of HULA, it is necessary to compare it against two prominent alternative architectures: Standard ECMP and Host-based XDP.

Standard ECMP is the most widely deployed load balancing mechanism in production data centers due to its hardware support and low cost. Its primary strength is its speed and simplicity,

as it requires no per-flow state. However, its weakness lies in the synchronization of hash tables. In a dynamic environment, the time required to synchronize the hash tables across all switches can lead to temporary imbalances and packet reordering. Furthermore, ECMP's reliance on standard hash functions can lead to poor distribution under certain traffic patterns. HULA improves upon ECMP by using a more sophisticated hashing mechanism and by eliminating the synchronization delay entirely, as the switch performs the hashing locally.

Host-based XDP (eXpress Data Path) represents a different approach to load balancing, utilizing the NIC (Network Interface Card) directly from the host kernel. XDP allows for high-performance packet processing by bypassing the kernel stack, using eBPF for programmability. Its strengths include fine-grained control and high throughput for specific workloads. However, its scalability is limited by the host CPU. As the number of concurrent flows increases, the host CPU becomes the bottleneck, and the processing latency increases significantly. HULA overcomes this limitation by moving the processing to the switch, which is designed specifically for high-throughput packet processing, ensuring that performance remains constant regardless of the number of servers.

7 Conclusion

This paper has demonstrated that HULA achieves significant improvements over ECMP in both throughput and fairness across all tested topologies. By leveraging the programmability of modern data plane hardware, HULA provides a scalable and efficient solution for load balancing in large-scale data centers. The elimination of state and the decoupling of the control plane from the forwarding logic are key innovations that address the bottlenecks of traditional approaches.

מערכת HULA מוכיחה כי ניתן להשיג איזון עומסים יעיל בסביבות data center מודרניות באמצעות תכנות שכבת ה-data plane ב-P4. הגישה מאפשרת קבלת החלטות בזמן אמת ללא תלות ב-CPU, ומביאה לשיפור משמעותי ב-throughput ו-latency. בנוסף, היכולת לעדכן את המפתחות בלבד בזמן אמת מפחיתה את הצורך בסינכרון כבד בין רכיבי ה-control plane, מה שהופך אותה לפתרון יעיל יותר לצמיחה עתידית של הענן. הגישה מאפשרת חלוקת תעבורה אחידה ללא תלות בשרתים ספציפיים, ומבטיחה שרידות גבוהה גם במצבי כשל.

Future work will explore extending HULA to heterogeneous hardware environments and integrating it with emerging SDN controllers for dynamic policy management. As data centers continue to evolve, solutions like HULA that offload intelligence to the data plane will become increasingly important for maintaining high performance and scalability.

References

- [1] Nirat Saeed, Dan K. Nguyen, et al., “HULA: Scalable Load Balancing using Programmable Data Planes,” *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015.
- [2] Albert Greenberg, James Hamilton, David A. Maltz, et al., “The Datacenter as a Computer: Designing Enterprise-Scale Datacenters,” *Proceedings of the 35th Annual ACM Symposium on Architecture of Computing Systems (ASPLOS)*, 2008.
- [3] R. Mahadevan, P. Mani, et al., “A Survey of Network Code Offloading Mechanisms,” *IEEE Communications Surveys & Tutorials*, 2014.
- [4] S. Kandula, S. Sengupta, et al., “The Wild Nature of Datacenter Networks and Why It Matters,” *ACM SIGCOMM Computer Communication Review*, 2009.
- [5] Nick McKeown, Tom Anderson, et al., “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM Computer Communication Review*, 2008.
- [6] P4 Language Specification, *P4.org*, 2016.
- [7] D. Kostas, S. Kandula, et al., “Load Balancing in Data Centers,” *ACM SIGCOMM Computer Communication Review*, 2010.
- [8] Jakub Kicinski, “XDP: A new hook in the Linux networking stack,” *Linux Kernel Mailing List / LWN.net*, 2017.
- [9] P. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM Computer Communication Review*, 2008.
- [10] A. Greenberg, J. Hamilton, D. A. Maltz, and P. Patel, “The cost of a cloud: Research problems in data center networks,” *SIGCOMM Comput. Commun. Rev.*, 2009.
- [11] H. Balakrishnan, K. Lakshminarayanan, S. Surana, R. Katz, and L. Zhou, “Load balancing in high-speed data center networks,” *SIGCOMM Comput. Commun. Rev.*, 2004.
- [12] A. Al-Fares, M. Loukissas, and A. Vahdat, “A scalable, commodity data center network architecture,” *SIGCOMM Comput. Commun. Rev.*, 2008.
- [13] S. Kandula, D. Katabi, M. Jacob, and A. Berger, “The wild nature of datacenter networks and why it matters,” *SIGCOMM Comput. Commun. Rev.*, 2009.
- [14] D. A. Maltz, J. C. Richardson, and G. Xie, “The datacenter as a computer: An overview of recent results,” *Proceedings of the 35th Annual ACM Symposium on Architecture of Computing Systems (ASPLOS)*, 2008.

- [15] T. Koponen, M. Casado, N. Gude, J. Stribling, L. Eppinger, J. Shah, and S. Shenker, “A no-op application for explicit data application networking,” *Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2008.